



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 03

Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

The state space means all the possible positions or situations that can exist in the environment. The search tree is the structure created by the search algorithm when it explores different possible paths from the starting point to the goal.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

The reached set is used to remember the positions that the agent has already visited. This helps the agent avoid visiting the same position again and again. Without the reached set, DFS may keep going through the same states repeatedly and can even get stuck in an infinite loop.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

BFS checks the nearest positions first and explores the grid level by level. In this grid, every movement has the same cost, so BFS can find the shortest path to the food. DFS goes deeply in one direction before checking other paths, so it may take a longer and more winding route even when a shorter path is available.

4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

BFS needs more memory because it keeps many possible states in memory at the same time while exploring level by level. This can become a problem in a very large grid. DFS usually uses less memory because it mainly follows one path deeply before trying another path. Therefore, DFS is generally more memory-efficient than BFS.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation

BFS

The screenshot shows the BFS simulation interface on the left and the code editor on the right. The simulation window displays a 10x10 grid with a red square at (0,0) and (0,9), a blue square at (2,2), and several purple circles representing obstacles. The text "Finished! Final Score: 255" is displayed at the bottom of the grid. A "Start Simulation" button is visible. The code editor shows the `SearchAgent` class in `agent.py` with the `sense_and_act` method. The terminal shows the command `python visual_grid_g` being executed.

DFS

The screenshot shows the DFS simulation interface on the left and the code editor on the right. The simulation window displays a 10x10 grid with a red square at (0,0) and (0,9), a blue square at (2,2), and several purple circles representing obstacles. The text "Finished! Final Score: 130" is displayed at the bottom of the grid. A "Start Simulation" button is visible. The code editor shows the `SearchAgent` class in `agent.py` with the `__init__` method. The terminal shows the command `python visual_grid_g` being executed.

The image shows a Python IDE with a grid-based search simulation on the left and code on the right.

Grid Simulation (Left):

- Grid size: 10x10.
- Start (Red Square): (0, 0).
- Goal (Blue Circle): (9, 9).
- Walls (Grey Squares): (2, 1), (3, 2), (3, 3), (4, 2), (4, 3), (5, 2), (5, 3), (6, 2), (6, 3), (7, 2), (7, 3), (8, 2), (8, 3), (9, 2), (9, 3).
- Obstacles (Purple Circles): (1, 4), (2, 4), (3, 4), (4, 4), (5, 4), (6, 4), (7, 4), (8, 4), (9, 4).
- Path (Red Squares): (0, 0) → (0, 1) → (0, 2) → (0, 3) → (0, 4) → (0, 5) → (0, 6) → (0, 7) → (0, 8) → (0, 9) → (1, 9) → (2, 9) → (3, 9) → (4, 9) → (5, 9) → (6, 9) → (7, 9) → (8, 9) → (9, 9).

Code (Right):

```
def __init__(self):  
    # Stores the sequence of planned actions.  
    self.plan = []  
  
    # Change this value to:  
    # 'BFS', 'DFS', or 'UCS'  
    self.active_algo = 'UCS'  
  
    # =====  
    # FIND VALID NEIGHBOURS  
    # =====  
    def get_neighbors(self, position, grid_size, wall):  
        """  
        Return all valid neighboring cells and the ac  
        required to reach each neighbor.  
        """  
        x, y = position
```

Terminal (Bottom):

```
PS C:\Users\ASUS\Desktop\IT3012---Practical-Base> git branch  
master  
PS C:\Users\ASUS\Desktop\IT3012---Practical-Base> python visual_grid_g  
ame.py  
PS C:\Users\ASUS\Desktop\IT3012---Practical-Base> python visual_grid_g  
ame.py  
PS C:\Users\ASUS\Desktop\IT3012---Practical-Base> python visual_grid_g  
ame.py  
PS C:\Users\ASUS\Desktop\IT3012---Practical-Base> python visual_grid_g  
ame.py
```

Output (Bottom):

Finished! Final Score: 300

Start Simulation